Assignment 2 : Simple Search Engine using Hadoop MapReduce

Course: Big Data - IU
Author: Firas Jolha
Deadline: April 3

Repository template

- https://github.com/firas-jolha/big-data-assignment2 (https://github.com/firas-jolha/big-data-assignment2)

Use the provided repository template and read all `README.md` files in all folders to understand how you should use the template. This is a mandatory step in this assignment.

Dataset

- A large corpus of plain text documents (e.g., news articles, Wikipedia pages, Stack Overflow posts).
  - https://www.kaggle.com/datasets/jjinho/wikipedia-20230701?select=a.parquet (https://www.kaggle.com/datasets/jjinho/wikipedia-20230701?select=a.parquet)
  - You can bring similar plain text documents if you have
    - The naming format of the text documents should be `<doc_id>_<doc_title>.txt` .

You do not need to download all the dataset files in the provided link. Check the tasks below to understand how to use it.

Agenda

- Assignment 2 : Simple Search Engine using Hadoop MapReduce
- Repository template
- Dataset
- Agenda
- Prerequisites
- Description
  - Search engine
  - TF/IDF
  - Best Match 25 (BM25)
- Data collection and preparation
- [50 points + 10 extra] Indexer tasks
- [50 points] Ranker tasks
- Report structure
- Instructions
- Plagiarism policy
- References

Prerequisites

- Docker and docker compose

Description

In this assignment, you need to implement a simple search engine with Hadoop MapReduce, Cassandra/ScyllaDB and Spark RDD. It should support the indexing, ranking and retrieval of plain text documents. The engine should retrieve documents relevant to a query given as an input text. The goal is to practice on using MapReduce, Cassandra/ScyllaDB and Spark for building such naive search engine and the complexity of the search is not the main performance gauge here.

You can introduce additional optimizations, but you have to feature them in the report and describe how they improve your software. Remember that your optimizations should not compromise the baseline search quality.

Search engine

The goal is to implement a simple search engine based on a naive text indexer and a document retriever. These components are often found in search engines. Conceptually, search engines were the first which tackled the problem of Big Data with the constraint of

low latency response. Imagine a typical search engine that has millions of documents in its index. Every second it receives hundreds to thousands of queries and requires to produce a list of the most relevant documents at sub-millisecond speed. In addition, it also needs to index new documents regularly. The problem of finding relevant information is one of the key problems in the field of Information Retrieval, which can be subdivided into two related tasks:

- Document indexing
- Query answering (Document retrieval)

While the second task is more related to real-time low latency processing, the indexing step can be done offline as batch processing. Hadoop MapReduce here will be used to do batch processing. It will create the index of a text corpus and store it in (distributed) Cassandra/ScyllaDB database (cluster). For the sake of this assignment, you are going to do a naive implementation of both tasks using MapReduce paradigm and Spark RDDs. Cassandra/ScyllaDB cluster here contains one node but you can scale it out if you have more resources.

The most common task in Information Retrieval (IR) is textual information retrieval. Whenever a user submits a query, the ranking engine should compute the set of the most relevant documents in its collection. To complete this task, the developer should determine how the documents and queries are represented, and define the measure of relevance of a query for a particular document.

TF/IDF

One of the most simple IR models is the TF/IDF model. It states that we need to count the occurrence of the search term in a given document.

Let’s take a small corpus composed of three documents as shown below.

```
d1 = cats and dogs are pets.
d2 = cats and dogs are pet animals though I prefer dogs. Dogs obey our commands, can b
d3 = Horses are also pets.
```

Let’s assume that the user’s query is “dogs” where the TF values are:

```
tf("dogs", d1) = 1
tf("dogs", d2) = 3
tf("dogs", d3) = 0
```

We compute *IDF* for the entire corpus and not just a single document. It measures the importance of a term ("dogs" in our case) across the entire corpus. It's the logarithm of the total number of documents in the corpus divided by the number of documents containing the term. So, the IDF score for our example is log(3/2) as "dogs" occur in 2 of the 3 documents.

Calculating TF-IDF simply boils down to multiplying the two values TF and IDF. So, the results show that document (d2) will be retrieved.

```
idf("dogs") = log(total docs N/number of docs containing the word "dogs") = log(3/2) =
```

Now we can calculate the tf-idf for each query term per document as follows.

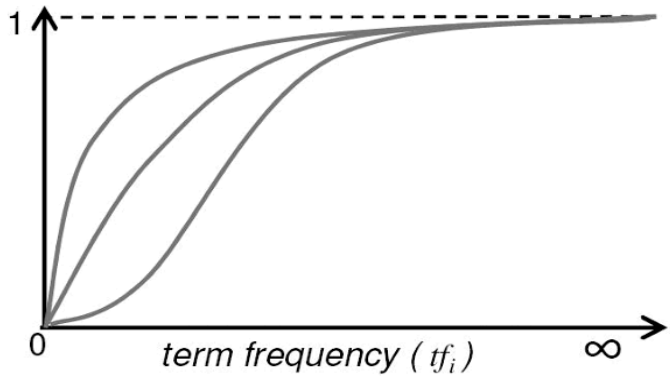
```
tf-idf("dogs", d1) = 1 * 0.176 = 0.176
tf-idf("dogs", d2) = 3 * 0.176 = 0.528
tf-idf("dogs", d3) = 0 * 0.176 = 0
```

There is a small problem however with our TF-IDF calculation. We haven't considered the length of our document. It matters how long a sentence is and we cannot ignore that. For example, though "dogs" occurs 3 times in the second document (d2), there are 25 words in it. But the first document(d1) only has 5 words. So let's **normalize** our TF-IDF by simply dividing it with the **document length**.

```
tf-idf("dogs", d1) = (1 * 0.176) / 5 = 0.0352
tf-idf("dogs", d2) = (3 * 0.176) / 25 = 0.0212
tf-idf("dogs", d3) = (0 * 0.176) / 4 = 0
```

Suddenly, our retrieval result now returns document (d1) instead of document (d2) as `tf-idf("dogs", d1) > tf-idf("dogs", d2)`. We can make it more sophisticated by including something called **saturation**.

Saturation is to penalize words that occur more in documents or words should get lesser weightage or value as they occur more and more in a document. Let's consider the second document, "Cats and dogs are pet animals though I prefer dogs. Dogs obey our commands, can be trained easily, and play with us all the time". What if the document keeps repeating the term, "dogs" over and over again? As the term frequency increases, we will give lesser and lesser weightage to terms as shown in the plot from the paper on BM25.



A plot of saturation of term frequency as it occurs more and more in a document.
Source: https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
(https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf)

Best Match 25 (BM25)

If we incorporate saturation and document length into TF-IDF, we will arrive at the BM algorithms. More specifically, we are interested in the most popular BM25 which incorporates saturation function and document length using this scary-looking formula:

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \log\left[\frac{N}{df(t)}\right] . \frac{(k_1 + 1).\, tf(t, d)}{k_1 .\, [(1 - b) + b.\, \frac{dl(d)}{dl_{avg}}] + tf(t, d)}$$

Where

- q: the user's query
- t: a term t in the user's query
- N : number of documents in the corpus
- $df(t)$: the document frequency or number of documents containing the term t.
- k_1, b: model's hyperparameters (e.g. $k_1 = 1, b = 0.75$)
- $tf(t, d)$: the term t frequency in the document d.
- $dl(d)$: the length of the document d.
- dl_{avg}: average document length.

To calculate BM25, we will have to include the average length of the document which is fairly easy for our toy corpus, which is $(5 + 25 + 4)/3 = 11.33$. k_1 and b are the hyperparameters chosen by us. But typically k_1 is 1 or 2 and b is 0.75. Let k_1=1 and $b = 0.75$ then $dl_{avg} = (5 + 25 + 4)/3 = 11.3$. We plug in the formula for BM25 and we get the following values.

```
BM25("dogs", d1) = 0.7818
BM25("dogs", d2) = 0.3093
BM25("dogs", d3) = 0
```

That's it! We have just calculated the BM25 for the term "dogs" on our toy corpus of three sentences. Now in this assignment, you need to implement the BM25 model.

Dear reader, notice that here we had only one term ("dogs") for the query but in general case you will have more than one term where you need to accumulate the BM25 scores for all query terms in order to obtain the BM25 score for a specific document in the corpus as written in the formula of BM25 above.

Data collection and preparation

In the whole assignment, it is not allowed to use Pandas or any package that operate on single machine. You should use PySpark for any thing related to data preparation.

1. Download one parquet file from the provided link and extract from it the data for at least 1000 documents/articles where each article/document is stored as a row in the parquet file.
2. The parquet file contains id, title, and text for each document. It also include categories but we are not interestd in it and you can drop it.
3. For each article, you should have an id, title and text content. Articles are plain text and stripped of all Wiki formatting syntax, including font styles, citations, links, etc.
4. When you create the documents, ensure that the documents have a unified naming format (`<doc_id>_<doc_title>.txt`). For example, `1_Art.txt` for an article whose id is 1 and its title is `Art` . Replace spaces with underscores `_` for the document titles. The id and title will help you to identify the document when you retrieve. Make sure that all documents contain some plain text content (size > 0) and not stored in a specific format like xml or json. Each file should have only plain text content (utf-8) and the file name contains the document id and the document title.
5. Put the parquet file in HDFS and create these documents and store them in the folder `/data` in HDFS.
6. These documents will be the input to the indexer that will be implemented using Hadoop MapReduce in the next section.
7. You need to prepare the documents for the first Hadoop MapReduce pipeline. Using PySpark RDD operations, read all documents in the folder `/data` in an RDD, transform the RDD (extract id, title and content of the documents) and store it in HDFS in `/input/data` as one partition. The output RDD file in `/input/data` should have the following content per line (separated by tab or some other character).

```
<doc_id>    <doc_title>    <doc_text>
```

You can use the following script to create documents from the provided parquet file:

```
# ./app/prepare_data.py
from pathvalidate import sanitize_filename
from tqdm import tqdm
from pyspark.sql import SparkSession


spark = SparkSession.builder \
    .appName('data preparation') \
    .master("local") \
    .config("spark.sql.parquet.enableVectorizedReader", "true") \
    .getOrCreate()


df = spark.read.parquet("/a.parquet")
n = 1000
df = df.select(['id', 'title', 'text']).sample(fraction=100 * n / df.count(), seed


def create_doc(row):
    filename = "data/" + sanitize_filename(str(row['id']) + "_" + row['title']).re
    with open(filename, "w") as f:
        f.write(row['text'])


df.foreach(create_doc)


# df.write.csv("/input/data", sep = "\t")
```

[50 points + 10 extra] Indexer tasks

These tasks are related to building a text indexer.

Before you start with these tasks, ensure that you have Hadoop cluster of at least 2 nodes (a master and a slave) running and ready to execute Hadoop MapReduce and Spark jobs.

In the whole assignment, it is not allowed to use Pandas or any package that operate on single machine. You should use PySpark for any thing related to data preparation.

1. **[20 points]** Using Hadoop MapReduce framework, build MapReduce pipelines that will receive input data for all documents, index them and stores the index data as one partition in HDFS (`/indexer`). You should have at least one MapReduce pipeline here but you may need to build more than one pipeline for the index. For each mapreduce pipeline, create `mapperx.py` and `reducerx.py` where `x` stands for the order of the pipeline and starts from 1. For example, the first pipeline will be `mapper1.py` and `reducer1.py` . For intermediate input/output, you can use a folder like `/tmp/indexer` in HDFS. This task should be done mainly using Hadoop Map Reduce. Spark RDD is not used here. The schema of output lines from Map-Reduce pipelines is not determined and based on your system design choices. However, you should minimally have output files for storing the vocabulary of the extracted terms, the index of the documents and the statistics required for calculating BM25 scores for the indexed documents. Allocate separate folders for the different pipelines in the path `/indexer` in HDFS. For example, you can use `/indexer/index` for storing the index. Add the details regarding system design choices and justification to the report as explained in the Report Structure section below. Write a script `create_index.sh` that will run all pipelines and creates the index for the input documents. This script accepts an argument for the path of the input data folder in HDFS to index all its files. By default it should read the files in the folder `/input/data` in HDFS.
2. **[25 points]** Write a script `store_index.sh` that will read the index data from HDFS and store it in Cassandra/ScyllaDB tables. You can use here `cassandra-driver` to load the data to tables or you can use PySpark with Connector (https://github.com/scylladb/spark-scylladb-connector (https://github.com/scylladb/spark-scylladb-connector)). We do not have any restriction here and using connector with Spark is the better option for loading big data. The schema of the data in Cassandra/ScyllaDB tables is not determined and based on your design choices but you should minimally have tables for storing the vocabulary of the extracted terms, the index of the documents and the statistics required for calculating BM25 scores for the indexed documents. Add these details to the report as explained in the Report Structure section below.
3. **[5 points]** Write a script `index.sh` that will create the index by running `create_index.sh` , then storing to Cassandra/ScyllaDB by running `store_index.sh` .
4. **[Optional: 10 extra]** Write a script `add_to_index.sh` to enable the user to pass a file from the local file system to index it. The important point here is to manage your index data in Cassandra/ScyllaDB tables when a new document got indexed.

[50 points] Ranker tasks

These tasks are related to building a Ranker.

In the whole assignment, it is not allowed to use Pandas or any package that operate on single machine. You should use PySpark for any thing related to data preparation.

1. **[15 points]** Build a PySpark application `query.py` that will read the user query as a text input from stdin and retrieves the list of top 10 relevant documents ranked using BM25. This should be done using PySpark RDD API. If you wanted to use Spark Dataframe API for some tasks, it is also fine. This script will calculate the BM25 metric for all query terms in all documents and retrieves the top 10 relevant documents. Here you need to read the index and vocabulary from Cassandra/ScyllaDB tables.
2. **[35 points]** Add all shell commands needed to run this PySpark application on the YARN cluster in fully-distributed environment in a script `search.sh`. This script will take the query as an argument from the user and retrieves the top 10 relevant documents. In the output, it is enough to display the document ids and titles.

We did not ask here to build/use a vector space model for the search engine but if you are interested you can do that. The documents are ranked based on their BM25 relevance scores for the user's query.

Report structure

The report should have minimaly the following sections:

1. **Methodology**: includes explanation of your design choices and approaches in implementing the search engine with sufficient details about all its components.
2. **Demonstration**: includes a guide on how to run your repository and FULLSCREEN screenshots about the successful indexing of 100 documents and successful execution of the search engine on some queries (e.g. two or three queries). Add your explanations for the retrieved results and reflections on your own findings.

Instructions

In this assignment, you need to submit all required files for running your search engine to your repository. In Moodle, you need to submit the link to the repository. You must follow the repository template shared with you. You can add more files or remove some files but

check below to see which files your repository should minimally have. You also need to submit the report `report.pdf`. In summary, you need to submit at least the following files in the repository:

- `mapreduce/mapper1.py`
  - mapper script for mapreduce pipeline
- `mapreduce/reducer1.py`
  - reducer script for mapreduce pipeline

> You should have at least one MapReduce pipeline but you may have more than one pipeline then submit all mapreduce script files in the folder `./app/mapreduce`.

- `create_index.sh`
  - creating vocabulary and index and storing them in HDFS.
- `store_index.sh`
  - For storing the index data in Cassandra/ScyllaDB tables
- `index.sh`
  - creating vocabulary and index and storing them in Cassandra/ScyllaDB.
- `query.py`
  - For searching in the engine given a user query and retrieving the list of top 10 relevant documents.
- `app.sh`
  - Starting the services
  - Running the indexer
  - Running the ranker for some queries
- `report.pdf`
  - Your report
- `requirements.txt`
  - Python dependencies
  - This does not determine which package manager you use but the requirements should be kept in this file
- `search.sh`
  - For running the search engine on a specific query
- `start-services.sh`
  - For starting the services of Hadoop

Notes:

- The assignment is **individual** and the student should submit only their own original code and report.
- The instructor will grade your submission as follows:
  - They will clone the repository in a local folder in a machine were Docker and Docker compose are installed.
  - They will access the directory where the repository is cloned.
  - They will run the command `docker compose up`.
    - We assume that you have configured the scripts in the repository to start services and install your project depndencies.
  - They may change the script `app.sh`
    - to add more test queries for the script `search.sh`.
    - to index more text files using the script `index.sh`

Plagiarism policy

It is **allowed** to use any support tools but not recommended to us GenAI tools for generating text or code. You can use these generative tools to proofread your report or check the syntax of the code. All solutions must be submitted individually. We will perform a plagiarism check on the code and report and you will be penalized if your submission is found to be plagiarized.

References

- https://www.ai-bites.net/tf-idf-and-bm25-for-rag-a-complete-guide/ (https://www.ai-bites.net/tf-idf-and-bm25-for-rag-a-complete-guide/)
- https://en.wikipedia.org/wiki/Okapi_BM25 (https://en.wikipedia.org/wiki/Okapi_BM25)